PARTIE 1 — Définitions Rapides (Théorie)

1. React – Définition

React est une bibliothèque JavaScript pour construire des interfaces utilisateur basées sur des composants.

- Architecture component-based
- DOM virtuel (Virtual DOM)
- Flux de données unidirectionnel

2. Composant

Deux types :

A) Functional Component (moderne)

```
function MyComponent() {
  return <h1>Hello</h1>;
}
```

B) Class Component (ancien)

```
class MyComponent extends React.Component {
  render() {
    return <h1>Hello</h1>;
  }
}
```

3. Props

- Données envoyées du parent vers l'enfant
- Lecture seule (immutable)

```
function Child({ name }) {
  return <p>{name}</p>;
}

<Child name="Youness" />
```

4. State

- Données internes au composant
- Mutable via setState ou useState

5. useState

Hook pour gérer le state dans functional component.

const [count, setCount] = useState(0);

- count → valeur
- setCount → fonction pour modifier

→ React class

- this.state.count
- this.setState({count:value})

6. useEffect

Permet de gérer les effets secondaires :

- Fetch API
- Timer
- Event listeners

```
useEffect(() => {
  console.log("Mounted");
}, []);
```

- [] → s'exécute au mount uniquement
- [value] → s'exécute quand value change

7. Lifecycle (Class Components)

→ Mount

- constructor()
- render()
- componentDidMount()

→ Update

- render()
- componentDidUpdate()

→ Unmount

- componentWillUnmount()

8. Formulaire en React

Toujours contrôler l'input avec state :

<input value={name} onChange={(e) => setName(e.target.value)} />

9. CRUD

CRUD =

- Create
- Read
- Update
- Delete

Sans API → manipulation d'un tableau local.
Avec API → fetch / axios.

10. React Router

React Router DOM permet de gérer la navigation SPA (Single Page Application) sans recharger la page.

Concepts importants :

- BrowserRouter
- Routes
- Route
- Link
- useParams

11. Redux Vanilla

- Store
- Action
- Reducer
- Dispatch

Flux :

Component → dispatch(action) → reducer → store update → UI update

12. Redux Toolkit

Simplifie Redux :

- configureStore
- createSlice

Plus besoin de switch-case manuel.

13. React Testing (Enzyme)

Permet :

- Shallow rendering
- Tester les props
- Tester les événements

PARTIE 2 — Exemples de Code (Révision Pratique)

Exemple 1 : Ajouter un étudiant

Question Examen :

Créer une application qui ajoute un étudiant dans une liste.

```jsx
import { useState } from "react";

function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState("");

  const addStudent = () => {
    const newStudent = {
      id: Date.now(),
      name: name
    };

    setStudents([...students, newStudent]);
    setName("");
  };

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Student name"
      />

      <button onClick={addStudent}>Add</button>

      <ul>
        {students.map((student) => (
          <li key={student.id}>{student.name}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Exemple 2 : Supprimer un étudiant

```
const deleteStudent = (id) => {
  setStudents(students.filter(student => student.id !== id));
};
Bouton :
<button onClick={() => deleteStudent(student.id)}>
  Delete
</button>
```

Exemple 3 : Update étudiant

```
const updateStudent = (id) => {
  const newList= students.map(student =>
    student.id === id
      ? { ...student, name: "Updated Name" }
      : student
  );

  setStudents(newList);
};
```

Exemple 4 : Fetch API avec useEffect

```jsx
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []);

  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

→ **Avec AXIOS**

```jsx
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  }, []);

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

Exemple 5 : Même chose en Class Component

```
class App extends React.Component {
  state = {
    users: []
  };

  componentDidMount() {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => this.setState({ users: data }));
  }

  render() {
    return (
      <div>
        {this.state.users.map(user => (
          <p key={user.id}>{user.name}</p>
        ))}
      </div>
    );
  }
}
```

Exemple 6 : React Router

→ App.js

```
import { Routes, Route, Link } from "react-router-dom";
import UsersPage from "./UsersPage";
import UserDetails from "./UserDetails";

function App() {
  return (
    <>
      <nav>
        <Link to="/users">Users</Link>
      </nav>

        <Route path="/users" element={<UsersPage />} />
        <Route path="/user/:id" element={<UserDetails />} />
      </Routes>
    </>
  );
}

export default App;
```

→ UsersPage.js

```
import { Link } from "react-router-dom";
import users from "../data/users"; // adjust path if needed

function UsersPage() {
  return (
    <div>
      <h1>Users Page</h1>

      {users.map((user) => (
        <div key={user.id}>
          <Link to={`/user/${user.id}`}>
            {user.name}
          </Link>
        </div>
      ))}
    </div>
  );
}

export default UsersPage;
```

→ UserPage.js

```
import { useParams } from "react-router-dom";

function UserPage() {
  const { id } = useParams();

  return <h2>User ID: {id}</h2>;
}
```

Exemple 7 : Redux Vanilla

→ Reducer.js

```
import { createStore } from "redux";

const initialState = { count: 0 };

function reducer(state = initialState, action) {
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    default:
      return state;
  }
}

const store = createStore(reducer);

export default store;
```

→ store.js

```
import { createStore } from "redux";
import reducer from "./reducer";

const store = createStore(reducer);

export default store;
```

→ index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import store from "./store";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

→ Utilisation

```jsx
import { useSelector, useDispatch } from "react-redux";

function App() {

  // 🔷 Lire la valeur du store
  const count = useSelector((state) => state.count);

  // 🔷 Envoyer une action
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>

      <button onClick={() => dispatch({ type: "INCREMENT" })}>
        Increment
      </button>
    </div>
  );
}

export default App;
```

Exemple 8 : Redux Toolkit

→ **StudentSlice.js**

```
import { createSlice } from "@reduxjs/toolkit";

const studentSlice = createSlice({
  name: "students",
  initialState: {
    students: []
  },
  reducers: {
    addStudent: (state, action) => {
      state.students.push(action.payload);
    },

    deleteStudent: (state, action) => {
      state.students = state.students.filter(
        student => student.id !== action.payload
      );
    }
  }
});

export const { addStudent, deleteStudent } = studentSlice.actions;
export default studentSlice.reducer;
```

→ Store.js

```
import { configureStore } from "@reduxjs/toolkit";
import studentReducer from "./features/studentSlice";

const store = configureStore({
  reducer: {
    students: studentReducer
  }
});

export default store;
```

→ Utilisation

```jsx
import { useSelector, useDispatch } from "react-redux";
import { deleteStudent } from "./features/studentSlice";

function Students() {
  const students = useSelector(
    state => state.students.students
  );

  const dispatch = useDispatch();

  return (
    <>
      <ul>
        {students.map(student => (
          <li key={student.id}>
            {student.name}
            <button onClick={() =>
              dispatch(deleteStudent(student.id))
            }>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </>
  );
}

export default Students;
```

→ REDUX + API

1. REDUX VANILA

npm install redux redux-thunk

```
import { createStore, applyMiddleware } from "redux";
import thunk from "redux-thunk";
import axios from "axios";

// 🔷 Async Action (Thunk)
export const fetchUsers = () => {
  return async (dispatch) => {
    dispatch({ type: "LOADING" });

    const res = await axios.get(
      "https://jsonplaceholder.typicode.com/users"
    );

    dispatch({
      type: "SET_USERS",
      payload: res.data
    });
  };
};


const initialState = {
  users: [],
  loading: false
};

function reducer(state = initialState, action) {
  switch (action.type) {
    case "LOADING":
      return { ...state, loading: true };

    case "SET_USERS":
      return { ...state, loading: false, users: action.payload };

    default:
      return state;
  }
}


const store = createStore(
  reducer,
  applyMiddleware(thunk)
);

export default store;
```

2. REDUX Toolkit

```
import { configureStore, createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import axios from "axios";

// 🔷 Async API
export const fetchUsers = createAsyncThunk(
  "users/fetch",
  async () => {
    const res = await axios.get(
      "https://jsonplaceholder.typicode.com/users"
    );
    return res.data;
  }
);

const userSlice = createSlice({
  name: "users",
  initialState: {
    users: [],
    loading: false
  },
  reducers: {},

  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.users = action.payload;
      });
  }
});

const store = configureStore({
  reducer: userSlice.reducer
});

export default store;
```

Exemple 9 : Test avec Enzyme

```
import { shallow } from "enzyme";
import App from "./App";

test("renders button", () => {
  const wrapper = shallow(<App />);
  expect(wrapper.find("button").length).toBe(1);
});
```

→ Commandes d’instalation REACT

1 Créer le projet React

Avec Create React App (CRA) :

npx create-react-app name-project

Avec Vite (plus rapide) :

npm create vite@latest my-app
cd my-app
npm install

---

2 React Router DOM

npm install react-router-dom

---

3 Redux Vanilla + Thunk

npm install redux react-redux redux-thunk

---

4 Redux Toolkit

npm install @reduxjs/toolkit react-redux

---

5 Axios (API requests)

npm install axios

---

6 Tests React

npm install --save-dev enzyme enzyme-adapter-react-18